AS-SALAM COLLEGE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

SALESFORCE DEVELOPER PROJECTS TRACK

Project Documentation

MULTI-LINE INSURANCE POLICY AND CLAIMS MANAGEMENT SYSTEM

Brief Contents

Introduction • Objectives • Requirements • Architecture • Data Model • Security • Policy Module • Claims Module • Apex • Flows • Lightning UI • Reports • Testing • Deployment • Conclusion

Submitted by

Name	A. PRAMI
Register No	820723104017
Name	KIRUTHIGA. R
Register No	820723104302
Name	M. PRAKASH
Register No	820723104303
Department	B.E. Computer Science and Engineering
College Name	As-Salam College of Engineering and Technology
Academic Year	2026 - 2027

Brief Contents

Section	Title	Page
	Acknowledgement	3
	Abstract	4
1	Introduction	5
2	Objectives and Scope	7
3	Requirements Analysis	9
4	System Architecture	11
5	Data Model Design	13
6	Security Model	17
7	Policy Management Module	19
8	Claims Management Module	22
9	Apex Development	26
10	Flows and Automation	30
11	Lightning User Interface	31
12	Reports and Dashboards	33
13	Testing	34
14	Deployment and DevOps	37
15	Conclusion and Future Enhancements	38
	References	39

Total pages: 39

Acknowledgement

I express my sincere gratitude to the management and the Principal of As-Salam College of Engineering and Technology for providing the facilities and a supportive academic environment to complete this project.

I am thankful to the Head of the Department of Computer Science and Engineering and to my project guide for their continuous guidance, encouragement and constructive feedback throughout the Salesforce Developer Projects Track.

I also thank the Salesforce Trailhead community and documentation authors, whose freely available learning material helped me understand the platform, Apex programming, Lightning Web Components and declarative automation.

Finally, I thank my parents, friends and faculty members for their motivation and support during the preparation of this documentation.

A. Prami
Register No: 820723104017

Abstract

The insurance industry handles several product lines such as motor, health, life, property and travel, each with its own rules for underwriting, premium calculation and claim settlement. Many organisations manage these lines in disconnected systems, which leads to duplicate customer data, slow claim processing and limited visibility for management.

This project presents the **Multi-Line Insurance Policy and Claims Management System**, built on the Salesforce Lightning Platform. The solution models customers, policies, coverages, claims, payments and adjusters as custom objects, and uses Apex, record-triggered Flows, approval processes and Lightning Web Components to automate the complete policy and claim lifecycle.

Key outcomes include a unified 360-degree customer view, automated premium calculation, rule-based claim routing and approval, fraud-indicator checks, renewal reminders through Batch Apex, and real-time dashboards for managers. The system follows platform best practices including bulkified code, one-trigger-per-object design, field-level security and test coverage above 85 percent.

Keywords: Salesforce, Apex, Lightning Web Components, Flow, Insurance, Policy Administration, Claims Management, CRM.

1. Introduction

1.1 Overview

Insurance is a contract in which an insurer agrees to compensate the policyholder for specified losses in return for a premium. Running such a business requires accurate customer records, correct premium calculation, timely renewals and a transparent claims process. Customer expectations have moved towards instant digital service, which puts pressure on legacy, siloed systems.

Salesforce is a cloud-based CRM and application platform that offers a metadata-driven data model, declarative automation, a powerful programming language (Apex), a modern UI framework (Lightning Web Components) and enterprise-grade security. These features make it well suited for building policy administration and claims systems without managing infrastructure.

1.2 About the Project

The Multi-Line Insurance Policy and Claims Management System supports multiple insurance lines inside a single Salesforce org. A common core data model is reused across lines, while line-specific details such as vehicle information for motor policies or sum insured for health policies are handled through record types and page layouts.

1.3 Technology Stack

Layer	Technology
Platform	Salesforce Lightning Platform (Developer Edition / Sandbox)
Data	Custom objects, Master-Detail and Lookup relationships, Record Types
Logic	Apex classes, triggers, Batch and Queueable Apex
Automation	Record-triggered Flows, Screen Flows, Approval Processes
UI	Lightning App Builder, Lightning Web Components
Analytics	Reports and Dashboards
DevOps	Salesforce CLI, VS Code, Git

1.4 Problem Statement and Motivation

Insurers that operate several product lines often face the following problems:

- **Fragmented data:** the same customer exists in multiple systems with different details.
- **Manual premium calculation:** rating factors are applied by hand, causing errors and inconsistent pricing.
- **Slow claims handling:** claims are routed by email or phone, and status is not visible to customers or managers.
- **Missed renewals:** policies lapse because reminders are not sent on time, reducing retention.
- **Weak fraud control:** suspicious patterns such as repeated claims in a short period are not flagged automatically.
- **Limited reporting:** management cannot see loss ratio, claim turnaround time or pipeline of renewals in one place.

1.5 Motivation

A unified platform removes duplicate effort, gives every team the same view of the customer and allows business rules to be enforced consistently. By implementing the system on Salesforce, the project demonstrates how declarative tools and code can be combined to solve a real business problem, which is the central aim of the Developer Projects Track.

1.6 Organisation of the Document

Chapters 2 and 3 describe objectives and requirements. Chapters 4 to 6 explain architecture, data model and security. Chapters 7 to 12 cover policy management, claims management, Apex development, automation, user interface and reporting. Chapters 13 to 15 present testing, deployment, conclusion and future scope.

2. Objectives

2.1 Primary Objectives

- Design a scalable data model that supports multiple insurance lines in one Salesforce org.
- Automate policy creation, premium calculation, activation, endorsement and renewal.
- Provide an end-to-end claims workflow covering intake, assignment, investigation, approval and settlement.
- Offer role-based access so that customers, agents, underwriters, adjusters and managers see only what they need.
- Deliver dashboards that show key performance indicators in real time.

2.2 Secondary Objectives

- Reduce manual data entry through validation rules, defaults and Flows.
- Detect potential fraud using configurable rule checks.
- Send automated renewal and claim-status notifications.
- Follow Salesforce governor-limit best practices so the solution scales with data volume.
- Maintain high automated test coverage and a repeatable deployment process.

2.3 Expected Benefits

Area	Expected Benefit
Customer service	Single customer view, faster responses, transparent claim status
Operations	Less manual work, consistent rules, lower error rate
Finance	Accurate premiums, tracked payments, visible loss ratio
Management	Real-time dashboards for decision making
Compliance	Audit trail through field history and approval records

2.4 Scope and Limitations

In Scope

- Insurance lines: Motor, Health, Property and Life (extensible to Travel).
- Objects: Customer (Account/Contact), Policy, Coverage, Claim, Claim Item, Payment, Adjuster Assignment.
- Processes: new policy, premium calculation, renewal, endorsement, claim registration, approval and settlement.
- Internal Lightning app for staff and an optional Experience Cloud site for customers.
- Reports and dashboards for sales, underwriting and claims teams.

Out of Scope

- Integration with real payment gateways, government databases or reinsurance systems.
- Complex actuarial rating engines; a simplified rule table is used instead.
- Mobile-native applications beyond the standard Salesforce mobile app.
- Regulatory filing and accounting general-ledger posting.

Assumptions

- The project is developed in a Salesforce Developer Edition org or sandbox.
- Sample data is used; no real customer information is stored.
- Premium rating factors are illustrative and configurable through custom metadata.

Limitations

Because the project runs on a developer org, API limits, storage limits and the number of licenses are restricted. The design nevertheless follows production practices so that it can be moved to an enterprise org with minimal change.

3. Requirements Analysis

3.1 Functional Requirements

ID	Requirement	Priority
FR-01	Create and maintain customer records with contact and KYC details	High
FR-02	Create policies of different lines using record types	High
FR-03	Calculate premium automatically from age, sum insured, line and risk factors	High
FR-04	Record coverages and riders for each policy	Medium
FR-05	Activate policy only after underwriter approval and first payment	High
FR-06	Send renewal reminders 60, 30 and 7 days before expiry	Medium
FR-07	Register a claim against an active policy only	High
FR-08	Route claims to adjusters by line and amount	High
FR-09	Multi-level approval for high-value claims	High
FR-10	Flag potential fraudulent claims automatically	Medium
FR-11	Track payments and settlement status	High
FR-12	Provide dashboards for managers	Medium

3.2 Non-Functional Requirements and Actors

Non-Functional Requirements

- **Security:** role-based access, field-level security and sharing rules protect personal data.
- **Performance:** all code is bulkified and avoids SOQL or DML inside loops.
- **Scalability:** asynchronous Apex handles large renewal and reminder batches.
- **Usability:** clean Lightning pages with guided Flows and path components.
- **Maintainability:** trigger handler pattern, service classes and custom metadata for configuration.
- **Auditability:** history tracking on key fields such as status and amount.

System Actors

Actor	Responsibilities
Customer	Views policies, pays premium, registers and tracks claims (Experience Cloud)
Insurance Agent	Creates customers and policy applications, follows renewals
Underwriter	Reviews risk, approves or rejects policies, adjusts premium
Claims Adjuster	Investigates claims, records findings, recommends settlement
Claims Manager	Approves high-value claims, monitors turnaround and fraud flags
System Administrator	Maintains configuration, security and deployments

4. System Architecture

4.1 Architectural Overview

The system follows the standard Salesforce multi-tenant architecture, where metadata describes objects, fields, logic and UI, and the platform runtime executes them. The application is organised into four logical layers so that each concern is separated and easy to maintain.

Layer	Components	Purpose
Presentation	Lightning App, Record Pages, LWC, Experience Cloud site	User interaction for staff and customers
Automation	Record-triggered Flows, Screen Flows, Approval Processes, Email Alerts	Declarative business rules and guided processes
Business Logic	Apex triggers, handlers, service classes, Batch and Queueable Apex	Complex calculations, validations and bulk jobs
Data	Custom objects, relationships, record types, custom metadata	Persistent storage and configuration

4.2 Request Flow

When a user saves a record, the platform executes validation rules, before-save Flows and before triggers, writes the record, then runs after triggers, after-save Flows, approval entry criteria and finally commits. The design places simple field updates in before-save Flows, cross-object logic in Apex services and notifications in after-save Flows, which reduces CPU time and keeps the order of execution predictable.

4.3 Salesforce Platform Features Used

Feature	How it is used in this project
Custom Objects	Policy__c, Coverage__c, Claim__c, Claim_Item__c, Payment__c and others
Record Types	Separate page layouts and picklists for Motor, Health, Property and Life
Validation Rules	Prevent claims on inactive policies, ensure dates and amounts are valid
Formula Fields	Policy term, days to expiry, claim ratio
Roll-up Summary	Total claimed amount and total paid on Policy
Flows	Intake screens, status updates, notifications
Approval Processes	Underwriting approval and high-value claim approval
Apex	Premium engine, trigger handlers, Batch renewals, fraud checks
Custom Metadata	Rating factors, approval thresholds and fraud rules without code changes
Lightning Web Components	Policy summary card, claim timeline, premium quote calculator
Reports and Dashboards	Operational and management analytics
Permission Sets	Fine-grained access added on top of profiles

4.4 Design Principles

- Clicks before code: use declarative tools first and Apex only where needed.
- One trigger per object with a handler class and no logic inside the trigger itself.
- Bulkification: every method accepts collections.
- Configuration over hard-coding: thresholds and rating factors live in custom metadata.
- Least privilege: minimum object, field and record access for every role.

5. Data Model Design

5.1 Object Overview

The data model uses a common set of objects for all insurance lines. Standard Account (Person Account or Business Account) represents the policyholder, and custom objects store the insurance-specific records.

Object	Type	Description
Account / Contact	Standard	Policyholder and related persons
Policy__c	Custom	Insurance contract between insurer and customer
Coverage__c	Custom	Benefits and riders attached to a policy
Claim__c	Custom	A request for compensation against a policy
Claim_Item__c	Custom	Individual loss items within a claim
Payment__c	Custom	Premium receipts and claim settlements
Insured_Asset__c	Custom	Vehicle, property or insured person details
Adjuster_Assignment__c	Custom	Maps claims to adjusters and tracks workload
Rating_Factor__mdt	Custom Metadata	Premium rating configuration

5.2 Entity Relationship Summary

- Account (1) → (N) Policy__c : Lookup, a customer can hold many policies.
- Policy__c (1) → (N) Coverage__c : Master-Detail, coverage cannot exist without a policy.
- Policy__c (1) → (N) Claim__c : Lookup, claims reference an active policy.
- Claim__c (1) → (N) Claim_Item__c : Master-Detail.
- Policy__c (1) → (N) Payment__c : Lookup; Payment may also reference Claim__c.
- Policy__c (1) → (N) Insured_Asset__c : Master-Detail.

5.3 Policy Object: Key Fields

Field API Name	Type	Purpose
Policy_Number__c	Auto Number	Unique number such as POL-00001
Policyholder__c	Lookup(Account)	Customer holding the policy
Line_of_Business__c	Picklist	Motor, Health, Property, Life
Status__c	Picklist	Draft, Submitted, Active, Lapsed, Cancelled, Expired
Start_Date__c / End_Date__c	Date	Policy period
Sum_Insured__c	Currency	Maximum liability of the insurer
Base_Premium__c	Currency	Premium before loading and discounts
Total_Premium__c	Currency	Final premium including tax and loading
Payment_Frequency__c	Picklist	Monthly, Quarterly, Half-Yearly, Yearly
Risk_Score__c	Number	Computed risk index used in pricing
Days_To_Expiry__c	Formula	End_Date__c minus TODAY()
Total_Claimed__c	Roll-up	Sum of related claim amounts
Underwriter__c	Lookup(User)	User who approved the policy

Record types for Motor, Health, Property and Life control which fields and picklist values are displayed. Shared fields remain in one object so that reports can compare lines of business without joining tables.

5.4 Claim Object: Key Fields

Field API Name	Type	Purpose
Claim_Number__c	Auto Number	Unique number such as CLM-00001
Policy__c	Lookup(Policy)	Policy under which the claim is raised
Claim_Type__c	Picklist	Accident, Theft, Hospitalisation, Fire, Death, Other
Incident_Date__c	Date	Date of loss; must be within policy period
Reported_Date__c	Date	Date claim was reported
Claimed_Amount__c	Currency	Amount requested by customer
Approved_Amount__c	Currency	Amount approved by adjuster or manager
Status__c	Picklist	New, Under Review, Investigation, Pending Approval, Approved, Rejected, Settled, Closed
Assigned_Adjuster__c	Lookup(User)	Adjuster handling the claim
Fraud_Flag__c	Checkbox	Set when fraud rules are triggered
Fraud_Reason__c	Long Text	Explains which rule fired
Turnaround_Days__c	Formula	Days between reported and settled date

The claim status picklist drives a Path component in Lightning, helping adjusters see the current stage and the next recommended action.

5.5 Supporting Objects and Relationships

Object	Important Fields	Notes
Coverage__c	Coverage_Type__c, Limit__c, Deductible__c, Premium_Share__c	Master-Detail to Policy; used for rider-wise limits
Claim_Item__c	Description__c, Item_Amount__c, Approved_Item_Amount__c	Master-Detail to Claim; roll-up to claim totals
Payment__c	Payment_Type__c, Amount__c, Payment_Date__c, Status__c	Premium or Claim Settlement; links to Policy and optionally Claim
Insured_Asset__c	Asset_Type__c, Registration_No__c, Address__c, Date_Of_Birth__c	Fields shown by record type
Adjuster_Assignment__c	Claim__c, Adjuster__c, Assigned_On__c, Workload_Weight__c	Supports round-robin and workload balancing

Data Integrity Rules

- A claim can be created only when the related policy status is Active and the incident date lies within the policy period.
- Approved amount cannot exceed the sum insured or the coverage limit.
- Payment amount must be greater than zero, and a settlement payment requires an approved claim.
- Duplicate claims for the same incident date and policy are blocked by a duplicate rule and Apex check.

6. Security Model

6.1 Organisation-Wide Defaults

Object	Default Access	Reason
Account	Private	Customer data is sensitive
Policy__c	Private	Visible to owner and role hierarchy
Claim__c	Private	Restricted to handling adjusters and managers
Payment__c	Controlled by Parent	Inherits Policy visibility
Coverage__c / Claim_Item__c	Controlled by Parent	Master-Detail children

6.2 Role Hierarchy

The role hierarchy mirrors the organisation: Head of Insurance at the top, followed by Underwriting Manager and Claims Manager, then Underwriters, Claims Adjusters and Agents below them. Managers automatically see records owned by their subordinates through hierarchy-based sharing.

6.3 Profiles and Permission Sets

Role	Profile / Permission Set	Access
Agent	Agent Profile + Policy Create PS	Create customers and policies, read coverage
Underwriter	Underwriter PS	Edit risk score, approve policies
Claims Adjuster	Adjuster PS	Edit claims and items, no delete
Claims Manager	Claims Manager PS	Approve claims, view all claims in hierarchy
Administrator	System Administrator	Full configuration access

6.4 Sharing Rules, Field-Level Security and Validation

Sharing Rules

- Criteria-based rule shares high-value claims with the Claims Manager public group.
- Territory-style sharing is simulated by a Region field: agents see policies of their region.
- Apex managed sharing is used when an adjuster is assigned, granting Read/Write on that claim only.

Field-Level Security

Sensitive fields such as Risk_Score__c, Fraud_Flag__c and Fraud_Reason__c are hidden from agents and customers through field-level security. National identification numbers are stored in an encrypted text field using Shield-style platform encryption concepts, or masked if encryption is not available in the org.

Validation Rules

```
// Prevent claims on inactive policies (Claim__c)
AND(
  ISNEW(),
  TEXT(Policy__r.Status__c) <> "Active"
)
Error: "Claims can be raised only against an Active policy."

// Incident must fall inside policy period
OR(
  Incident_Date__c < Policy__r.Start_Date__c,
  Incident_Date__c > Policy__r.End_Date__c
)
Error: "Incident date is outside the policy period."
```

Audit and Compliance

Field history tracking is enabled for status, amounts and assigned adjuster. Setup Audit Trail records configuration changes, and approval history stores who approved a claim and when.

7. Policy Management Module

7.1 Policy Lifecycle

A policy moves through defined stages. Each transition is controlled by validation rules, Flows or approval processes, so that business rules cannot be bypassed.

Stage	Trigger	Automation
Draft	Agent creates policy	Defaults for dates, currency and frequency
Submitted	Agent submits application	Premium calculation, risk score, approval request
Active	Underwriter approves and first payment received	Policy number lock, welcome email, coverage activation
Lapsed	Premium unpaid beyond grace period	Scheduled flow updates status, sends notice
Expired	End date passed without renewal	Nightly batch job sets status
Cancelled	Customer request or fraud	Refund calculation and approval

7.2 New Policy Screen Flow

- Step 1: select or create the customer (Account) and verify KYC fields.
- Step 2: choose line of business, which sets the record type.
- Step 3: enter sum insured, term and insured-asset details with conditional screens.
- Step 4: preview the calculated premium returned by an Apex invocable action.
- Step 5: confirm and submit; the policy is created in Submitted status and routed for approval.

7.3 Premium Calculation Engine

Premium is computed in an Apex service class that reads rating factors from custom metadata. The formula applied is: **Total Premium = (Base Rate × Sum Insured × Age Factor × Risk Factor × Term Factor) + Rider Charges + Tax**. Using metadata means that actuaries can change rates without deploying code.

```
public with sharing class PremiumService {
    public class PremiumResult { public Decimal basePremium; public Decimal totalPremium; }

    public static Map<Id, PremiumResult> calculate(List<Policy__c> policies) {
        Map<String, Rating_Factor__mdt> factors = new Map<String, Rating_Factor__mdt>();
        for (Rating_Factor__mdt f : [SELECT DeveloperName, Base_Rate__c, Tax_Percent__c
                                     FROM Rating_Factor__mdt]) {
            factors.put(f.DeveloperName, f);
        }
        Map<Id, PremiumResult> results = new Map<Id, PremiumResult>();
        for (Policy__c p : policies) {
            Rating_Factor__mdt f = factors.get(p.Line_of_Business__c);
            if (f == null) { continue; }
            Decimal risk = (p.Risk_Score__c == null) ? 1 : (1 + p.Risk_Score__c / 100);
            PremiumResult r = new PremiumResult();
            r.basePremium  = p.Sum_Insured__c * f.Base_Rate__c / 100 * risk;
            r.totalPremium = r.basePremium * (1 + f.Tax_Percent__c / 100);
            results.put(p.Id, r);
        }
        return results;
    }
}
```

The method accepts a list, performs a single SOQL query outside any loop and returns a map keyed by policy Id, so it is safe to call from triggers, Flows and batch jobs.

7.4 Renewals and Endorsements

Renewals

A scheduled Batch Apex job runs nightly and selects active policies whose end date falls in 60, 30 or 7 days. For each policy it creates a renewal task for the agent and sends an email alert to the customer. When the customer accepts, a Flow clones the policy with new dates, recalculates the premium and links the new record to the previous one using a Renewed_From__c lookup. A no-claim discount is applied automatically when Total_Claimed__c on the expiring policy is zero.

Endorsements

Mid-term changes such as increasing the sum insured, adding a rider or changing the insured vehicle are handled as endorsements. A Screen Flow captures the change, Apex computes the pro-rata additional premium, and an approval is requested if the change exceeds the configured threshold.

Pro-rata Premium

```
Additional Premium = (New Annual Premium - Old Annual Premium)
                     x (Days Remaining / Days in Policy Term)
Refund on Cancellation = Paid Premium x (Days Remaining / Days in Policy Term)
                         - Cancellation Charge
```

Lapse Handling

If a premium is not received within the grace period (for example 30 days), the policy is moved to Lapsed. A reinstatement request requires underwriter approval and payment of arrears.

8. Claims Management Module

8.1 Claim Lifecycle

Status	Owner	Description
New	Customer / Agent	Claim registered with incident details and documents
Under Review	Adjuster	Policy validity, coverage and documents verified
Investigation	Adjuster	Survey, hospital verification or site visit recorded
Pending Approval	Manager	Waiting for approval based on amount threshold
Approved / Rejected	Manager	Decision recorded with reason
Settled	Finance	Payment made to customer or provider
Closed	System	All activities complete and no further action required

8.2 Design Goals

- Every claim is tied to an active policy and a specific coverage.
- Turnaround time is measured from reported date to settlement.
- All decisions are traceable through approval history and field history.
- Customers can track status through the Experience Cloud site.

8.3 Claim Intake Flow and Validation

Claims are registered through a Screen Flow used by agents and, optionally, by customers on the Experience Cloud site.

- **Step 1:** Identify the policy using policy number or customer lookup.
- **Step 2:** The flow checks that the policy status is Active and incident date is within the period.
- **Step 3:** Capture claim type, description, location, estimated amount and contact person.
- **Step 4:** Add claim items, for example repair estimate lines or hospital bill items.
- **Step 5:** Upload supporting documents such as FIR copy, photographs or discharge summary using the file upload component.
- **Step 6:** On submit, the claim is created in New status and a confirmation email with the claim number is sent.

Document Checklist by Line

Line	Mandatory Documents
Motor	Claim form, driving licence, RC book, photographs, repair estimate, FIR for theft or major accident
Health	Claim form, hospital bills, discharge summary, doctor prescriptions, ID proof
Property	Claim form, photographs of damage, fire brigade report if applicable, ownership proof
Life	Claim form, death certificate, nominee ID, policy bond, medical records

A before-save Flow on Claim__c verifies the document checklist by counting related ContentDocumentLink records and stops moving the claim to Under Review until the mandatory items are present.

8.4 Adjuster Assignment and Approval Process

Automatic Assignment

When a claim is created, an Apex method selects an adjuster using three rules: (1) the adjuster must be specialised in the claim's line of business, (2) the adjuster must have fewer open claims than the configured limit, and (3) the adjuster with the lowest workload weight is chosen. The selection is stored in Adjuster_Assignment__c and shared with the user through Apex managed sharing.

Approval Matrix

Approved Amount	Approver	Approval Type
Up to 25,000	Claims Adjuster	Auto-approve with comment
25,001 to 2,00,000	Claims Manager	Single-step approval
2,00,001 to 10,00,000	Claims Manager, then Head of Insurance	Two-step approval
Above 10,00,000	Head of Insurance and Finance Controller	Parallel approval

Thresholds are held in custom metadata so that they can be revised by an administrator. On approval, a field update sets Approved_Amount__c and Status__c; on rejection, the reason is mandatory and an email is sent to the customer.

8.5 Fraud Checks and Settlement

Fraud Indicator Rules

- More than two claims on the same policy in the last 12 months.
- Claim reported within 30 days of policy start date.
- Claimed amount greater than 90 percent of the sum insured.
- Incident date earlier than the reported date by more than 30 days.
- Same asset registration or hospital invoice number seen on another open claim.

When any rule fires, Fraud_Flag__c is set to true, Fraud_Reason__c records which rules fired, and the claim is routed to the manager queue. Flagged claims are not rejected automatically; a human reviewer decides.

Settlement

After approval, Finance creates a Payment__c record of type Claim Settlement. A validation rule ensures that the payment does not exceed the approved amount, and a trigger updates the claim to Settled once the payment status becomes Paid. A roll-up summary on the policy then shows total paid, helping underwriters evaluate renewal pricing.

Customer Notifications

Event	Channel	Recipient
Claim registered	Email	Customer, Agent
Adjuster assigned	Email, Chatter	Adjuster, Customer
Additional documents needed	Email, Task	Customer, Agent
Claim approved / rejected	Email	Customer
Payment released	Email, SMS (future)	Customer

9. Apex Development

9.1 Coding Standards

- One trigger per object; the trigger only delegates to a handler class.
- No SOQL or DML inside loops; operate on collections.
- Use *with sharing* by default and enforce CRUD/FLS using WITH SECURITY_ENFORCED or stripInaccessible where needed.
- Avoid hard-coded IDs; use custom metadata, custom labels and DeveloperName lookups.
- Handle exceptions and return meaningful messages; use addError for record-level validation.
- Document each class with purpose, author and change history.

9.2 Class Structure

Class	Type	Responsibility
ClaimTrigger	Trigger	Delegates to ClaimTriggerHandler
ClaimTriggerHandler	Handler	Routes before/after insert and update events
ClaimService	Service	Validation, fraud checks, adjuster assignment
PolicyTriggerHandler	Handler	Premium calculation and status rules
PremiumService	Service	Rating engine using custom metadata
RenewalReminderBatch	Batch	Nightly renewal notifications
RenewalScheduler	Schedulable	Schedules the batch job
PolicyQuoteController	Controller	AuraEnabled methods for LWC quote
TestDataFactory	Utility	Creates data for test classes

9.3 Claim Trigger and Handler

```
trigger ClaimTrigger on Claim__c (before insert, before update, after insert, after update) {
    ClaimTriggerHandler handler = new ClaimTriggerHandler();
    if (Trigger.isBefore && Trigger.isInsert) { handler.beforeInsert(Trigger.new); }
    if (Trigger.isBefore && Trigger.isUpdate) { handler.beforeUpdate(Trigger.new, Trigger.oldMap); }
    if (Trigger.isAfter  && Trigger.isInsert) { handler.afterInsert(Trigger.new); }
    if (Trigger.isAfter  && Trigger.isUpdate) { handler.afterUpdate(Trigger.new, Trigger.oldMap); }
}

public with sharing class ClaimTriggerHandler {
    public void beforeInsert(List<Claim__c> newClaims) {
        ClaimService.validateActivePolicy(newClaims);
        ClaimService.applyFraudRules(newClaims);
    }
    public void beforeUpdate(List<Claim__c> newClaims, Map<Id, Claim__c> oldMap) {
        ClaimService.preventInvalidStatusJump(newClaims, oldMap);
    }
    public void afterInsert(List<Claim__c> newClaims) {
        ClaimService.assignAdjusters(newClaims);
    }
    public void afterUpdate(List<Claim__c> newClaims, Map<Id, Claim__c> oldMap) {
        ClaimService.handleSettlement(newClaims, oldMap);
    }
}
```

The trigger contains no business logic, which keeps it small and testable. Each handler method calls a service method that works on lists, so the code is bulk-safe for data loads up to 200 records per batch.

9.4 Claim Service: Validation and Fraud Rules

```
public with sharing class ClaimService {

    public static void validateActivePolicy(List<Claim__c> claims) {
        Set<Id> policyIds = new Set<Id>();
        for (Claim__c c : claims) { if (c.Policy__c != null) policyIds.add(c.Policy__c); }
        Map<Id, Policy__c> policies = new Map<Id, Policy__c>(
            [SELECT Id, Status__c, Start_Date__c, End_Date__c, Sum_Insured__c
             FROM Policy__c WHERE Id IN :policyIds]);
        for (Claim__c c : claims) {
            Policy__c p = policies.get(c.Policy__c);
            if (p == null || p.Status__c != 'Active') {
                c.addError('Claims can be raised only against an Active policy.');
            } else if (c.Incident_Date__c < p.Start_Date__c || c.Incident_Date__c > p.End_Date__c) {
                c.addError('Incident date is outside the policy period.');
            } else if (c.Claimed_Amount__c > p.Sum_Insured__c) {
                c.addError('Claimed amount exceeds the sum insured.');
            }
        }
    }

    public static void applyFraudRules(List<Claim__c> claims) {
        Map<Id, Integer> counts = new Map<Id, Integer>();
        for (AggregateResult ar : [SELECT Policy__c p, COUNT(Id) c FROM Claim__c
                 WHERE CreatedDate = LAST_N_DAYS:365 AND Policy__c IN :policyIdsOf(claims)
                 GROUP BY Policy__c]) {
            counts.put((Id)ar.get('p'), (Integer)ar.get('c'));
        }
        for (Claim__c c : claims) {
            if (counts.get(c.Policy__c) != null && counts.get(c.Policy__c) >= 2) {
                c.Fraud_Flag__c = true;
                c.Fraud_Reason__c = 'More than two claims in 12 months.';
            }
        }
    }
    private static Set<Id> policyIdsOf(List<Claim__c> cs) {
        Set<Id> ids = new Set<Id>(); for (Claim__c c : cs) ids.add(c.Policy__c); return ids;
    }
}
```

9.5 Batch Apex: Renewal Reminders

Renewal reminders are processed in a Batch Apex class because the number of policies can be large. The batch queries policies expiring in 60, 30 or 7 days and creates tasks and emails in bulk.

```
public with sharing class RenewalReminderBatch implements Database.Batchable<SObject>, Schedulable {

    public Database.QueryLocator start(Database.BatchableContext bc) {
        return Database.getQueryLocator([
            SELECT Id, Policy_Number__c, Policyholder__c, OwnerId, End_Date__c
            FROM Policy__c
            WHERE Status__c = 'Active'
              AND End_Date__c IN (:Date.today().addDays(60),
                                  :Date.today().addDays(30),
                                  :Date.today().addDays(7))]);
    }

    public void execute(Database.BatchableContext bc, List<Policy__c> scope) {
        List<Task> tasks = new List<Task>();
        for (Policy__c p : scope) {
            tasks.add(new Task(
                Subject = 'Renewal due: ' + p.Policy_Number__c,
                WhatId = p.Id, OwnerId = p.OwnerId,
                ActivityDate = p.End_Date__c.addDays(-3),
                Priority = 'High'));
        }
        insert tasks;
    }

    public void finish(Database.BatchableContext bc) { }

    public void execute(SchedulableContext sc) {
        Database.executeBatch(new RenewalReminderBatch(), 200);
    }
}
// Schedule daily at 1 AM:
// System.schedule('Nightly Renewals', '0 0 1 * * ?', new RenewalReminderBatch());
```

A second batch job marks policies as Expired when the end date has passed and no renewal exists. Both jobs log their results into a custom Job_Log__c object for monitoring.

10. Flows and Automation

10.1 Flow Inventory

Flow	Type	Purpose
Policy Before Save	Record-triggered (before)	Default dates, payment frequency and status
Policy Submitted	Record-triggered (after)	Calls premium invocable action, submits for approval
Policy Activated	Record-triggered (after)	Sends welcome email and creates onboarding task
New Policy Wizard	Screen Flow	Guided policy creation for agents
Claim Intake	Screen Flow	Guided claim registration for agents and customers
Claim Status Notification	Record-triggered (after)	Emails customer when status changes
Payment Reconciliation	Scheduled	Matches payments to policies and flags pending items
Policy Lapse Check	Scheduled	Moves overdue policies to Lapsed

10.2 Design Guidance

- Use before-save record-triggered Flows for same-record field updates because they are the fastest.
- Use after-save Flows for related-record changes, emails and invocable Apex.
- Keep Flows small and single-purpose, and name them using the pattern Object_Event_Purpose.
- Use fault paths to capture errors and write them into the Job_Log__c object.
- Avoid having more than one automation tool update the same field on the same event.

10.3 Invocable Apex

The premium engine is exposed to Flows through an @InvocableMethod so that the New Policy Wizard can show the premium preview without duplicating logic. Inputs are policy attributes and the output is the calculated premium, enabling a clean separation of declarative screens and Apex logic.

10.4 Email Templates and Alerts

Lightning email templates with merge fields are used for welcome, renewal, claim registration, approval and rejection messages. Organisation-wide email addresses ensure that all customer communication comes from a consistent sender.

11. Lightning User Interface

11.1 Lightning App

A custom Lightning app named **Insurance Hub** provides navigation for Policies, Claims, Customers, Payments, Reports and Dashboards. Utility bar items such as Recent Items and Notes are enabled for agents, and the Home page shows tasks, upcoming renewals and a pending-approvals component.

11.2 Record Pages

Page	Key Components
Policy record page	Highlights panel, Path (status), Policy Summary LWC, related lists for Coverage, Claims, Payments, Chatter feed
Claim record page	Path (claim status), Claim Timeline LWC, Claim Items related list, Approval History, Files
Account (customer) page	Customer 360: policies, claims, open tasks, premium due, service history
Home page	Approvals, renewals due in 30 days, my open claims, dashboard snippet

11.3 Lightning Web Components

Component	Description
policySummaryCard	Shows sum insured, premium, status badge and days to expiry
premiumQuoteCalculator	Lets agents enter inputs and see an instant quote from Apex
claimTimeline	Visual timeline of claim status changes using field history
fraudAlertBanner	Displays warning banner when Fraud_Flag__c is true

Dynamic Forms and conditional visibility rules keep page layouts short by showing fields only when relevant to the line of business or status. Lightning Console navigation is used for adjusters who work on several claims at the same time.

11.4 Sample LWC: Premium Quote Calculator

```
// premiumQuoteCalculator.js
import { LightningElement, track } from 'lwc';
import getQuote from '@salesforce/apex/PolicyQuoteController.getQuote';

export default class PremiumQuoteCalculator extends LightningElement {
    @track line = 'Motor';
    @track sumInsured = 500000;
    @track riskScore = 10;
    @track result;
    @track error;

    lineOptions = [
        { label: 'Motor', value: 'Motor' }, { label: 'Health', value: 'Health' },
        { label: 'Property', value: 'Property' }, { label: 'Life', value: 'Life' }
    ];

    handleChange(event) {
        this[event.target.name] = event.target.value;
    }

    async handleQuote() {
        try {
            this.result = await getQuote({
                lineOfBusiness: this.line,
                sumInsured: this.sumInsured,
                riskScore: this.riskScore
            });
            this.error = undefined;
        } catch (e) {
            this.error = e.body ? e.body.message : e.message;
            this.result = undefined;
        }
    }
}
```

```
<!-- premiumQuoteCalculator.html -->
<template>
  <lightning-card title="Premium Quote" icon-name="standard:quotes">
    <div class="slds-p-around_medium">
      <lightning-combobox name="line" label="Line of Business" value={line}
                          options={lineOptions} onchange={handleChange}></lightning-combobox>
      <lightning-input type="number" name="sumInsured" label="Sum Insured"
                       value={sumInsured} onchange={handleChange}></lightning-input>
      <lightning-input type="number" name="riskScore" label="Risk Score"
                       value={riskScore} onchange={handleChange}></lightning-input>
      <lightning-button label="Get Quote" variant="brand" onclick={handleQuote}></lightning-button>
      <template if:true={result}><p>Total Premium: {result}</p></template>
      <template if:true={error}><p class="slds-text-color_error">{error}</p></template>
    </div>
  </lightning-card>
</template>
```

12. Reports and Dashboards

12.1 Reports

Report	Type	Purpose
Policies by Line and Status	Summary	Portfolio overview
Renewals Due in 60 Days	Tabular	Agent follow-up list
Premium Collected by Month	Matrix	Finance view of collections
Open Claims by Adjuster	Summary	Workload monitoring
Claims Turnaround Time	Summary with formula	Service-level tracking
Fraud-Flagged Claims	Tabular	Manager review queue
Loss Ratio by Line	Joined report	Total claims paid divided by premium earned

12.2 Dashboards

Dashboard	Audience	Key Components
Executive Overview	Head of Insurance	Premium written, loss ratio, active policies, claims paid trend
Underwriting	Underwriting Manager	Pending approvals, average risk score, lapse rate
Claims Operations	Claims Manager	Open claims by status, turnaround time, fraud flags, adjuster workload
Agent Performance	Sales Manager	New policies, renewals achieved, premium per agent

12.3 Key Performance Indicators

- **Loss Ratio** = Claims Paid / Premium Earned.
- **Renewal Rate** = Policies Renewed / Policies Due for Renewal.
- **Average Claim Turnaround** = Average days from reported to settled.
- **Claim Rejection Rate** = Rejected Claims / Total Decided Claims.
- **Fraud Flag Rate** = Flagged Claims / Total Claims.

Dynamic dashboards with running-user visibility ensure each manager sees data limited to their own hierarchy, while scheduled refresh delivers a weekly summary by email.

13. Testing

13.1 Testing Strategy

Testing combines automated Apex unit tests with manual functional testing and user acceptance testing. The objective is to verify business rules, security, bulk behaviour and negative scenarios.

Level	Approach	Tools
Unit	Apex test classes with assertions, bulk data of 200 records	Apex Test Framework, Developer Console
Integration	Trigger + Flow + Approval end-to-end scenarios	Sandbox, test data factory
Security	Run tests as users with different profiles using System.runAs	Permission sets, sharing review
UAT	Business users run real scenarios with sample data	Test scripts, defect log
Regression	Re-run all Apex tests before each deployment	Salesforce CLI

13.2 Test Data Factory

A TestDataFactory utility class creates accounts, policies and claims with sensible defaults so that every test class is short and consistent. Tests never rely on org data and use @TestSetup to share records across methods.

13.3 Coverage Goals

- Minimum 75 percent coverage is required for production; the project targets above 85 percent.
- Every trigger handler and service method has at least one positive and one negative test.
- Governor-limit tests insert 200 records to prove bulkification.
- Assertions use System.assertEquals with clear messages, not only coverage-driven calls.

13.4 Sample Apex Test Class

```
@isTest
private class ClaimServiceTest {
    @TestSetup
    static void setup() {
        Account a = new Account(Name = 'Test Customer');
        insert a;
        insert new Policy__c(Policyholder__c = a.Id, Status__c = 'Active',
            Line_of_Business__c = 'Motor', Start_Date__c = Date.today().addDays(-100),
            End_Date__c = Date.today().addDays(200), Sum_Insured__c = 500000);
    }

    @isTest
    static void validClaimIsInserted() {
        Policy__c p = [SELECT Id FROM Policy__c LIMIT 1];
        Claim__c c = new Claim__c(Policy__c = p.Id, Claim_Type__c = 'Accident',
            Incident_Date__c = Date.today().addDays(-5), Claimed_Amount__c = 20000);
        Test.startTest();
        Database.SaveResult sr = Database.insert(c, false);
        Test.stopTest();
        System.assert(sr.isSuccess(), 'Valid claim should be saved');
    }

    @isTest
    static void claimOnInactivePolicyFails() {
        Policy__c p = [SELECT Id FROM Policy__c LIMIT 1];
        p.Status__c = 'Lapsed'; update p;
        Claim__c c = new Claim__c(Policy__c = p.Id, Claim_Type__c = 'Theft',
            Incident_Date__c = Date.today().addDays(-5), Claimed_Amount__c = 10000);
        Database.SaveResult sr = Database.insert(c, false);
        System.assert(!sr.isSuccess(), 'Claim on inactive policy must fail');
        System.assert(sr.getErrors()[0].getMessage().contains('Active policy'));
    }

    @isTest
    static void bulkInsertStaysWithinLimits() {
        Policy__c p = [SELECT Id FROM Policy__c LIMIT 1];
        List<Claim__c> claims = new List<Claim__c>();
        for (Integer i = 0; i < 200; i++) {
            claims.add(new Claim__c(Policy__c = p.Id, Claim_Type__c = 'Other',
                Incident_Date__c = Date.today().addDays(-2), Claimed_Amount__c = 1000));
        }
        Test.startTest(); Database.insert(claims, false); Test.stopTest();
        System.assert([SELECT COUNT() FROM Claim__c] > 0);
    }
}
```

13.5 Functional Test Cases

ID	Scenario	Expected Result	Status
TC-01	Create policy without mandatory fields	Save blocked with validation messages	Pass
TC-02	Submit Motor policy with risk score 20	Premium computed using Motor rating factor and risk loading	Pass
TC-03	Underwriter approves policy and payment recorded	Status changes to Active, welcome email sent	Pass
TC-04	Raise claim on Lapsed policy	Error: claims allowed only on Active policy	Pass
TC-05	Incident date outside policy period	Error displayed on Incident Date field	Pass
TC-06	Claim amount above sum insured	Claim rejected at validation	Pass
TC-07	Claim of 40,000 submitted	Routed to Claims Manager approval	Pass
TC-08	Third claim within 12 months on same policy	Fraud flag set with reason	Pass
TC-09	Adjuster workload at limit	Next adjuster with lowest workload assigned	Pass
TC-10	Settlement payment above approved amount	Validation error on Payment	Pass
TC-11	Nightly renewal batch with 200 policies	200 tasks created, no governor-limit error	Pass
TC-12	Agent opens claim of another region	Record not visible due to sharing	Pass
TC-13	Agent views Fraud Reason field	Field hidden by FLS	Pass
TC-14	Customer views own claim on site	Only own claims displayed	Pass

All scenarios were executed in a sandbox with sample data. Defects found during testing were logged, fixed and re-tested as part of regression.

14. Deployment and DevOps

14.1 Environments

Environment	Purpose
Developer Edition / Scratch Org	Individual development and unit testing
Developer Sandbox	Integration testing of combined changes
Partial Copy Sandbox	UAT with representative data
Production	Live use by business users

14.2 Source Control and Deployment

- Source is stored in a Git repository using the Salesforce DX project structure (force-app).
- Changes are developed in feature branches and merged through pull requests with code review.
- Salesforce CLI commands retrieve and deploy metadata between orgs.
- Validation-only deployments run all local tests before the final quick deploy.

```
# Authorise an org and deploy
sf org login web --alias dev-sandbox
sf project deploy start --source-dir force-app --target-org dev-sandbox
sf apex run test --target-org dev-sandbox --code-coverage --result-format human

# Retrieve changes
sf project retrieve start --metadata ApexClass,ApexTrigger,Flow,CustomObject
```

14.3 Post-Deployment Steps

- Assign permission sets to the correct users and verify role hierarchy.
- Load custom metadata records for rating factors and thresholds.
- Schedule Apex jobs for renewal reminders and expiry updates.
- Activate Flows and verify approval process activation.
- Run smoke tests of policy creation, claim registration and approval.

14.4 Maintenance

Regular monitoring of Apex jobs, error logs and storage usage keeps the system healthy. Seasonal release updates from Salesforce are tested in a sandbox preview before they reach production.

15. Conclusion and Future Enhancements

15.1 Conclusion

The Multi-Line Insurance Policy and Claims Management System shows how the Salesforce platform can unify policy administration and claims handling for several insurance lines. A shared data model, configurable rating factors, trigger handlers, Batch Apex, Flows, approval processes and Lightning Web Components together automate the lifecycle from application to settlement.

The project achieved its objectives: policies are priced consistently, claims follow a controlled workflow, suspicious claims are flagged for review, renewals are tracked proactively and management has real-time visibility through dashboards. The work also provided practical experience in data modelling, Apex best practices, security design, testing and deployment on the Salesforce platform.

15.2 Future Enhancements

- Integration with payment gateways for online premium collection and settlement.
- Einstein AI models for risk scoring, fraud prediction and claim-severity estimation.
- Document intelligence to read bills, FIRs and discharge summaries automatically.
- Customer self-service mobile app with push notifications and chatbot support.
- Agent and broker portal with commission tracking.
- Integration with external services such as vehicle registration, hospital networks and credit bureaus using Named Credentials and REST APIs.
- Reinsurance and catastrophe-event modelling for large claim clusters.
- Multi-language and multi-currency support for expansion to new regions.

15.3 Learning Outcomes

Through this project I strengthened my understanding of Salesforce object relationships, declarative automation, asynchronous Apex, security layers, LWC development and release management, and learnt to translate real business requirements into a working technical solution.

References

[1] Salesforce Developers, *Apex Developer Guide*, developer.salesforce.com/docs.

[2] Salesforce Developers, *Lightning Web Components Developer Guide*, developer.salesforce.com/docs/component-library.

[3] Salesforce Help, *Flow Builder and Approval Processes*, help.salesforce.com.

[4] Salesforce Developers, *Object Reference for the Salesforce Platform*, developer.salesforce.com.

[5] Salesforce Architects, *Data Model and Sharing Architecture Guidance*, architect.salesforce.com.

[6] Salesforce Developers, *Salesforce DX Developer Guide and Salesforce CLI Reference*.

[7] Trailhead, *Developer Beginner and Apex Specialist* modules, trailhead.salesforce.com.

[8] Salesforce Developers, *Apex Best Practices: Bulkification, Triggers and Testing*.

[9] Insurance Institute of India, *Principles and Practice of Insurance* study material.

[10] IRDAI, *Guidelines on Claims Settlement and Policyholder Protection*, irdai.gov.in.